



Instituto Politécnico Nacional

Escuela Superior de Computo



Base de Datos

Practica 1

Ejercicio 3

P R E S E N T A

Espinosa Gómez David Enrique

Estrada Sanchez Emiliano

Mora Acosta Pablo

3CV2

A 18 de septiembre de 2026

Introducción

Las bases de datos son uno de los pilares de la informática moderna, porque permiten organizar, almacenar y recuperar grandes volúmenes de información de manera eficiente y confiable. Casi cualquier aplicación actual, desde un sistema bancario hasta una red social, depende de un sistema gestor de bases de datos (SGBD) para funcionar bien. Este documento explica los conceptos de la Unidad Temática I: empieza con la diferencia entre dato e información y termina con la clasificación de los modelos de datos. Para ello se comparan las posturas de varios autores especializados y se explica cómo se pasó de los archivos tradicionales a los gestores actuales.

Qué es una base de datos y qué la hace útil

De los datos a la información

Antes de hablar de bases de datos conviene separar dos ideas. Un dato es un valor aislado que, sin contexto, no dice nada: el número 21 o la cadena 20260918 no comunican algo por sí solos. La información aparece cuando uno o varios datos se procesan y se interpretan dentro de un contexto, de modo que sirven para tomar decisiones (Elmasri & Navathe, 2017). Si al 21 se le asocia el atributo «edad de una persona», ya significa algo; y si a un 38.5 se le agrega que es la temperatura de un paciente en cierta fecha, ya es información útil: el paciente tiene fiebre.

Tres maneras de definir una base de datos

Los autores no la definen igual. Date (2004) la ve como un sistema computarizado cuyo propósito general es almacenar información y permitir que los usuarios la consulten y la actualicen cuando lo necesiten; para él, el sistema completo incluye los datos, el hardware, el software y las personas que interactúan con él. Elmasri y Navathe (2017) la definen como una colección de datos relacionados y lógicamente coherentes, con un significado implícito, que se diseña, se construye y se llena con datos para un propósito específico y que representa un aspecto del mundo real, llamado minimundo o universo de discurso; si algo cambia en ese minimundo, el cambio debe reflejarse en la base de datos. Silberschatz et al. (2019), por su parte, hablan de un conjunto de datos interrelacionados junto con los programas para acceder a ellos y manipularlos, para ofrecer un entorno cómodo y eficiente donde guardar y recuperar la información.

Al compararlas se nota que cada una mira algo distinto. Elmasri y Navathe se fijan en lo que los datos representan: lo importante es que reflejen la realidad con coherencia y respeten las reglas del negocio, más que en el equipo que los guarda. Silberschatz y sus colaboradores tienen un enfoque más práctico, de ingeniería, porque unen la estructura de los datos con los componentes que hacen que las consultas se respondan rápido en un sistema real (procesador de consultas, búferes, rutinas de optimización). Date tiene la visión más amplia, ya que incluye al hardware y a las personas. Aun así, las tres definiciones coinciden en algo esencial: una base de datos no es una simple acumulación de datos sueltos, sino un conjunto organizado y con propósito, hecho para consultarse y mantenerse a lo largo del tiempo.

Qué debe cumplir una buena base de datos

Lo que distingue a una base de datos bien diseñada de un simple conjunto de archivos son ocho características:

- **Integración.** Reúne en un solo repositorio lógico los datos de toda la organización, así que distintos subsistemas consultan la misma información sin depósitos aislados.
- **Persistencia.** Los datos siguen guardados en almacenamiento no volátil aunque la aplicación que los creó se cierre o el equipo se apague (Silberschatz et al., 2019).
- **Redundancia controlada.** Se evita repetir un mismo dato sin necesidad, por ejemplo con la normalización. Si se permite cierta duplicación planeada por rendimiento, el gestor se encarga de mantenerla sincronizada.
- **Integridad.** La información debe ser exacta y consistente. Para lograrlo se usan restricciones de dominio (tipos de dato válidos), de entidad (llaves primarias únicas y no nulas) y referenciales (llaves foráneas que apuntan a registros que sí existen).
- **Independencia de datos.** Los cambios en la estructura lógica o física no obligan a reescribir las aplicaciones que consultan la base.
- **Seguridad.** Cada usuario solo puede hacer lo que su rol permite, mediante autenticación, autorización por roles (RBAC) y cifrado.
- **Concurrencia.** Varios usuarios trabajan al mismo tiempo sin provocar anomalías como lecturas sucias, escrituras perdidas o lecturas no repetibles, gracias a las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad).
- **Recuperación.** Si hay una falla eléctrica o una caída del servidor, el sistema regresa a un estado válido apoyándose en bitácoras (*write-ahead logging*) y puntos de control (*checkpoints*) (Coronel & Morris, 2019).

Por qué los archivos tradicionales se quedaron cortos

Antes de que se consolidaran los SGBD, a finales de la década de 1960, las organizaciones manejaban su información con archivos independientes: cada aplicación, escrita por lo general en COBOL, Fortran o C, administraba sus propios archivos en cinta magnética o en disco. Ese esquema tenía tres problemas grandes.

El primero era la redundancia y la inconsistencia. Como cada departamento hacía sus propios programas, un mismo dato, por ejemplo la dirección de un cliente, se guardaba en varios archivos (el de ventas, el de facturación, etc.). Cuando el cliente se mudaba, un área actualizaba su archivo y las demás no, así que coexistían varias «verdades» contradictorias (Connolly & Begg, 2015). El segundo era la dependencia entre programas y datos: la estructura física de los registros, con la longitud de los campos y sus tipos de dato, estaba metida en el código de cada aplicación. Si había que ampliar el código postal de 5 a 9 caracteres, era necesario localizar, modificar, recompilar y probar todos los programas que usaban ese archivo. El tercero era la dificultad para hacer consultas nuevas: los datos estaban aislados en formatos incompatibles, y un reporte que nadie había previsto obligaba a escribir un programa de extracción desde cero.

Estas limitaciones (falta de independencia, dificultad para compartir datos y ausencia de control centralizado) motivaron el desarrollo de los SGBD. Un SGBD centraliza el almacenamiento, pone una capa de abstracción entre los datos y los programas y ofrece un lenguaje declarativo de acceso, lo que permite administrar la información de forma unificada y consistente (Date, 2004).

Quiénes trabajan con una base de datos

Alrededor de una base de datos interactúan cuatro tipos de usuarios, cada uno con responsabilidades y permisos distintos. El diseñador (lógico y físico) conversa con las personas del negocio para reunir los requerimientos y define el esquema: en la etapa lógica establece las entidades, sus atributos y sus relaciones, y en la física decide cómo se almacenarán los datos. Suele tener permisos amplios de DDL en los entornos de desarrollo, pero no accede a los datos sensibles de producción. El administrador de la base de datos (DBA) es la máxima autoridad sobre los recursos de la base: concede permisos, vigila el rendimiento, cuida la seguridad, procura que el servicio esté disponible todo el tiempo, ajusta parámetros de memoria (*tuning*) y se encarga de la replicación y de los respaldos. Por eso tiene privilegios de superusuario y acceso al catálogo del sistema (Coronel & Morris, 2019).

El programador de aplicaciones construye las aplicaciones web, móviles o de escritorio que usan la base de datos, con código embebido y consultas SQL o llamadas a un ORM, y en ellas implementa la lógica de negocio. Sus privilegios se limitan a manipular datos (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) en esquemas delimitados, y no puede modificar el esquema en producción. Por último, el usuario final, paramétrico o casual, como un cajero, un médico o un analista, consume la base con interfaces gráficas o formularios. No necesita conocer SQL ni la estructura de las tablas, y la aplicación controla sus permisos (Elmasri & Navathe, 2017).

Cómo se construye una base de datos

El desarrollo de una base de datos sigue un ciclo de vida de seis etapas sucesivas, con retroalimentación entre ellas, y cada etapa deja un entregable (Connolly & Begg, 2015):

1. **Planificación y factibilidad.** Se delimita el alcance del proyecto, se analiza si es viable técnica, económica y operativamente, y se eligen de forma preliminar las herramientas. Entregable: documento de términos de referencia y estudio de factibilidad.
2. **Análisis de requerimientos.** Se hacen entrevistas, se revisan los formatos que ya se usan y se levantan las necesidades de información y las reglas de negocio. Entregable: especificación de requerimientos de software (SRS) y matriz de reglas de negocio.
3. **Diseño.** Tiene tres fases. El diseño conceptual es un modelo abstracto, independiente del SGBD, que se representa con un diagrama entidad-relación. El diseño lógico lo traduce a un esquema relacional y lo normaliza (hasta la tercera forma normal) para eliminar redundancias e inconsistencias. En el diseño físico se eligen los tipos de datos del SGBD, los índices, las particiones y el almacenamiento. Entregables: diagrama E-R, esquema relacional normalizado y diccionario de datos.
4. **Implementación y prototipado.** Se escriben scripts DDL que crean las tablas, vistas, disparadores (*triggers*) y funciones almacenadas en el SGBD elegido. Entregable: scripts SQL de creación del esquema (por ejemplo, `schema.sql`) versionados en Git.

5. **Carga, migración y pruebas.** Se cargan los datos históricos, se depura la información y se prueba el sistema (estrés, integridad referencial y concurrencia) frente a los requisitos originales. Entregable: reporte de pruebas y base de datos poblada y validada.
6. **Operación, afinamiento y mantenimiento.** Ya en producción se monitorea el rendimiento, se aplican parches de seguridad, se ejecutan las políticas de respaldo y se adapta el esquema a nuevos requerimientos. Entregables: bitácora de auditoría, plan de recuperación ante desastres (DRP), reportes de afinamiento y los manuales de operación.

El sistema gestor de bases de datos por dentro

Los módulos del SGBD

Un SGBD está formado por varios módulos que trabajan de manera coordinada (Silberschatz et al., 2019). El gestor de almacenamiento y búferes se comunica con el sistema de archivos del sistema operativo para leer y escribir bloques de datos e índices. También administra el espacio en disco y el *buffer pool*, la memoria caché donde se guardan las páginas de disco que más se usan, con lo que aumenta la velocidad de lectura y escritura. El procesador y optimizador de consultas recibe las sentencias SQL, que son declarativas (indican qué se quiere obtener, no cómo), y hace su análisis léxico, sintáctico y semántico. Luego, un optimizador basado en costos evalúa varios planes de ejecución y elige el que consume menos CPU y menos lecturas de disco.

El gestor de transacciones y control de concurrencia asegura que las operaciones sean atómicas y aisladas aunque varios usuarios accedan a los mismos datos a la vez, ya sea con un administrador de bloqueos (*lock manager*) o con control de concurrencia multiversión (MVCC). Por último, el gestor de recuperación anota cada cambio en la bitácora de transacciones (WAL) antes de que se escriba en el disco principal, de modo que después de una falla puede rehacer (*REDO*) o deshacer (*UNDO*) las transacciones que quedaron a medias.

Los lenguajes del SGBD

Según la operación, el SGBD ofrece distintos lenguajes (Elmasri & Navathe, 2017):

- **DDL (Lenguaje de Definición de Datos).** Crea, modifica o elimina las estructuras de la base de datos (`CREATE`, `ALTER`, `DROP`, `TRUNCATE`).
- **DML (Lenguaje de Manipulación de Datos).** Consulta, inserta, actualiza y elimina la información guardada (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).
- **DCL (Lenguaje de Control de Datos).** Administra la seguridad, los permisos y los roles sobre los objetos de la base (`GRANT`, `REVOKE`).
- **TCL (Lenguaje de Control de Transacciones).** Confirma o revierte un conjunto de operaciones agrupadas como una sola transacción, para proteger la integridad de los datos ante errores o interrupciones (`COMMIT`, `ROLLBACK`, `SAVEPOINT`).

Cómo se clasifican los SGBD

Por el modelo de datos en que se basan, los sistemas relacionales (RDBMS) organizan la información en tablas de filas y columnas relacionadas mediante claves, y son el enfoque

dominante en las aplicaciones empresariales tradicionales; algunos ejemplos son PostgreSQL, MariaDB y Oracle. Los no relacionales (NoSQL) surgieron para los casos en que el volumen, la velocidad o la variedad de los datos hace poco práctico el modelo tabular rígido. Entre ellos están los documentales (MongoDB), los de clave-valor (Redis), los columnares (Apache Cassandra), los de grafos (Neo4j) y también los orientados a objetos, que guardan la información como objetos con atributos y comportamientos, de forma parecida a como se maneja en los lenguajes de programación orientados a objetos (Coronel & Morris, 2019).

También se pueden clasificar por el número de usuarios que soportan, ya sea monousuario (SQLite) o multiusuario (por ejemplo, PostgreSQL o CockroachDB), y por su arquitectura de despliegue: centralizados en un solo nodo, distribuidos (homogéneos o heterogéneos) o nativos de la nube (*cloud-native*, *serverless*) (Elmasri & Navathe, 2017).

Arquitectura del sistema: niveles y capas

Los tres niveles de abstracción

La arquitectura de tres niveles la propuso el comité ANSI/X3/SPARC en 1975 para lograr la independencia de datos, y organiza el sistema en tres capas de abstracción (Connolly & Begg, 2015):

- **Nivel externo.** Es el más cercano a los usuarios. Define vistas personalizadas, de modo que cada perfil solo ve y opera los datos que necesita para su trabajo, sin enfrentarse a la complejidad del resto de la base.
- **Nivel conceptual (o lógico).** Representa la base de datos completa tal como la percibe la organización en su conjunto: entidades, atributos, relaciones, operaciones y restricciones de integridad. No depende del hardware ni de cómo se guarde en disco.
- **Nivel interno (o físico).** Es el más bajo y describe cómo se almacenan realmente los datos: estructuras de bloques, apuntadores, algoritmos de indexación (árboles B, GIN, GiST), compresión y particionamiento.

Independencia de datos

Esta arquitectura hace posibles dos tipos de independencia (Date, 2004). La independencia lógica permite cambiar el esquema conceptual, por ejemplo agregar una entidad nueva o un atributo a una tabla, sin modificar los esquemas externos ni las aplicaciones que no usan lo que cambió. La independencia física permite cambiar el esquema interno, como migrar los archivos a un disco SSD más rápido, reorganizar el factor de llenado de las páginas o crear un índice nuevo en una columna que se consulta mucho, sin tocar el esquema conceptual, las vistas ni las aplicaciones.

Dónde se ejecuta el sistema: centralizado o cliente-servidor

En la arquitectura centralizada, todo el procesamiento (la interfaz, la lógica de la aplicación y el motor del SGBD) se ejecutaba en un mismo equipo, normalmente un mainframe, al que los usuarios entraban de forma directa o remota. La arquitectura cliente-servidor, en cambio, reparte el trabajo. En su versión de dos capas, el cliente ejecuta la aplicación gráfica y se conecta por TCP/IP con el servidor de base de datos mediante controladores como JDBC u ODBC. En la de

tres capas, que es la más moderna, el cliente es una interfaz ligera (un navegador web o una app móvil); la capa intermedia, el servidor de aplicaciones o *backend*, procesa las reglas del negocio; y la tercera capa es el SGBD, que se ocupa solo de la persistencia y de la consistencia de las transacciones. Esa capa intermedia separa la interfaz del motor de datos y facilita que el sistema escale (Silberschatz et al., 2019).

Modelos para representar los datos

Para qué sirve un modelo de datos

Un modelo de datos es una herramienta de abstracción y de comunicación. Reúne conceptos, notaciones gráficas y reglas que permiten describir los datos de un dominio, sus relaciones, su significado y sus restricciones, y con ello representar de forma simplificada los objetos, eventos y relaciones del mundo real que le importan a una organización, sin capturar todos los detalles posibles. Gracias a esa abstracción, los usuarios, analistas, diseñadores y programadores comparten una misma representación de los datos, aunque cada quien los use desde su propia perspectiva (Elmasri & Navathe, 2017).

Clasificación de los modelos de datos

Los modelos se pueden clasificar desde dos perspectivas que se complementan (Coronel & Morris, 2019). La primera es su nivel de abstracción:

- **Modelos conceptuales (de alto nivel).** Representan la realidad como la percibe el usuario, sin detalles técnicos ni dependencia de software comercial. El más conocido es el modelo entidad-relación (E-R) de Peter Chen y sus extensiones.
- **Modelos lógicos o de representación.** Ya incorporan estructuras que un SGBD puede implementar, aunque siguen sin depender de un producto específico. Aquí entran el modelo relacional, basado en las relaciones tabulares de Edgar F. Codd, el orientado a objetos y los modelos NoSQL de documentos JSON.
- **Modelos físicos (de bajo nivel).** Describen cómo se acomodan los registros en las páginas de almacenamiento, el ordenamiento por *hash*, los *clusters* y las rutas de acceso.

La segunda perspectiva es la estructura con la que organizan las relaciones entre los datos:

- **Jerárquico.** Organiza la información en un árbol en el que cada nodo hijo tiene un solo padre (relaciones 1:N estrictas). Quedó obsoleto porque no puede representar relaciones complejas de muchos a muchos (N:M).
- **En red (CODASYL).** Forma un grafo en el que un registro puede pertenecer a varios conjuntos mediante punteros físicos. Se dejó de usar porque mantenerlo era muy complicado.
- **Relacional.** Estructura la información en tablas bidimensionales: las filas son tuplas, las columnas son atributos y las tablas se vinculan por valores en común (llaves foráneas). Domina la industria por su base matemática y su sencillez declarativa.
- **Orientado a objetos.** Representa la información como objetos con atributos y métodos, parecido a los conceptos de la programación orientada a objetos.

- **NoSQL y multimodelo.** Paradigmas recientes (clave-valor, documentos, familias de columnas) que flexibilizan el esquema estricto y permiten escalar horizontalmente en entornos distribuidos.

Referencias

- Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6.^a ed., ed. global). Pearson.
- Coronel, C., & Morris, S. (2019). *Database systems: Design, implementation, & management* (13.^a ed.). Cengage Learning.
- Date, C. J. (2004). *An introduction to database systems* (8.^a ed.). Pearson/Addison-Wesley.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of database systems* (7.^a ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7.^a ed.). McGraw-Hill Education.